

Experiment No. 1

Name: Sunny

Section: B

Implementation of Fundamental Image Processing Operations using Python and OpenCV

Python / OpenCV Code

```
# Experiment No. 1
# Implementation of Fundamental Image Processing Operations
# using Python and OpenCV

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# -----
# STEP 1: LOAD IMAGE
# -----

image_path = "image.jpg"

img = cv2.imread(x=image_path)

if img is None:
    print("ERROR: Image could not be loaded.")
    print("Make sure image.jpg is in the same folder as labsheet1.py")
    exit()

print("Image loaded successfully!")

# -----
# STEP 2: DISPLAY ORIGINAL IMAGE
# -----

# OpenCV reads image in BGR format
# Matplotlib displays image in RGB format
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(img_rgb)
plt.title("Original image")
plt.axis("off")
plt.show()

# -----
# STEP 3: EXAMINE IMAGE PROPERTIES
# -----

height, width, channels = img.shape

print('\n=====')
print("      IMAGE PROPERTIES")
print('=====')

print("Width      :", width, "pixels")
print("Height     :", height, "pixels")
print("Channels    :", channels)
print("Resolution  :", width, "x", height)
print("Data Type   :", img.dtype)
print("Total Pixels :", height * width)

# -----
# STEP 4: SAVE IMAGE IN DIFFERENT FORMATS
```

```

# -----

jpeg_file = "output_image.jpg"
png_file = "output_image.png"

cv2.imwrite(jpeg_file, img)
cv2.imwrite(png_file, img)

print("\n=====")
print("      IMAGE SAVING")
print("=====")

print("JPEG saved as :", jpeg_file)
print("PNG saved as  :", png_file)

# Check file sizes

jpeg_size = os.path.getsize(jpeg_file)
png_size = os.path.getsize(png_file)

print("\nJPEG size :", jpeg_size, "bytes")
print("PNG size  :", png_size, "bytes")

# -----
# STEP 5: COLOR SPACE CONVERSION
# -----

# RGB
rgb_image = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Grayscale
gray_image = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# HSV
hsv_image = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# LAB
lab_image = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

# -----
# STEP 6: CONVERT HSV AND LAB FOR DISPLAY
# -----

# Convert HSV back to RGB for visualization
hsv_rgb = cv2.cvtColor(hsv_image, cv2.COLOR_HSV2RGB)

# Convert LAB back to RGB for visualization
lab_rgb = cv2.cvtColor(lab_image, cv2.COLOR_LAB2RGB)

# -----
# STEP 7: DISPLAY ALL COLOR SPACES
# -----

plt.figure(figsize=(12, 8))

# Original / RGB
plt.subplot(2, 2, 1)
plt.imshow(rgb_image)
plt.title("RGB Image")
plt.axis("off")

# Grayscale
plt.subplot(2, 2, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

# HSV
plt.subplot(2, 2, 3)
plt.imshow(hsv_rgb)
plt.title("HSV Image")
plt.axis("off")

# LAB
plt.subplot(2, 2, 4)
plt.imshow(lab_rgb)
plt.title("LAB Image")
plt.axis("off")
plt.tight_layout()

```

```

plt.show()

# -----
# STEP 8: COLOR SPACE INFORMATION
# -----

print("\n=====")
print("      COLOR SPACE INFORMATION")
print("=====")

print("Original BGR shape :", img.shape)
print("RGB shape          :", rgb_image.shape)
print("Grayscale shape     :", gray_image.shape)
print("HSV shape           :", hsv_image.shape)
print("LAB shape           :", lab_image.shape)

# -----
# STEP 9: DISPLAY PIXEL VALUES
# -----

x = 100
y = 100

if y < height and x < width:

    print("\n=====")
    print("      PIXEL VALUES")
    print("=====")

    print("Pixel position : (100,100)")

    print("BGR          :", img[y, x])
    print("RGB          :", rgb_image[y, x])
    print("HSV          :", hsv_image[y, x])
    print("LAB          :", lab_image[y, x])
    print("Grayscale    :", gray_image[y, x])

# -----
# EXPERIMENT COMPLETED
# -----

print("\n=====")
print(" Experiment completed successfully!")
print("=====")

```

Output Photo 1



Output Photo 2



Output Photo 3



Output Photo 4



Output Photo 5



Output Photo 6



Output Photo 7



Output Photo 8



Output Photo 9



Output Photo 10

